RESUMEN TEÓRICO · UA1

JavaScript: lo esencial de la teoría

Qué es JavaScript, qué tipo de lenguaje es, cómo maneja los tipos, sus paradigmas, los errores y cómo llega el código a la página

Módulo	CSTI12004 Programación web en el entorno cliente
Unidad de aprendizaje	UA1 · Elaboración de componentes de software
Capacidad	1.1 Elaborar componentes de software mediante el lenguaje JavaScript, según los estándares de desarrollo vigentes.
Persona facilitadora	Giovanni Antonio Coto Calderón

1. ¿Qué es JavaScript?

JavaScript es el lenguaje de programación de la web: es el **único lenguaje que todos los navegadores ejecutan directamente**. Con él, una página deja de ser solo para leer y empieza a responder a lo que hace la persona usuaria.

Dato	Detalle
Origen	Lo creó Brendan Eich en 1995, en la empresa Netscape.
Nombre	No tiene relación con Java: el nombre fue una decisión comercial de la época.
Estándar	Se llama ECMAScript (documento ECMA-262). Sale una edición por año; la vigente es ECMAScript 2026.
Dónde se ejecuta	En el navegador (Chrome, Edge, Firefox, Safari) y también fuera de él con Node.js, que usaremos en la UA4.
Motor	Cada navegador tiene un programa que lee y ejecuta el código: V8 en Chrome y Edge, SpiderMonkey en Firefox.

2. Las tres capas de una página web

Su sitio Casa Tomillo ya tiene dos capas, que hizo en el módulo de Codificación de páginas web. JavaScript le agrega la tercera.

Capa	Lenguaje	Responde a	En Casa Tomillo
Estructura	HTML	¿Qué hay en la página?	La tabla de talleres de cursos.html
Presentación	CSS	¿Cómo se ve?	Colores y letras de css/estilos.css
Comportamiento	JavaScript	¿Qué hace?	Calcular un precio o responder al botón Enviar de contacto.html

Cada capa va en su propio archivo: así como el CSS va en **css/estilos.css**, el JavaScript va en **js/app.js**.

3. ¿Qué tipo de lenguaje es?

JavaScript es un lenguaje **de alto nivel** e **interpretado**: se escribe con palabras cercanas al idioma humano y el navegador lo ejecuta tal como está, sin un paso previo de compilación.

- **Alto nivel:** no se manejan detalles de la máquina, como la memoria.
- **Interpretado:** basta con guardar el archivo y recargar la página. No se genera un .exe.
- **Dato extra:** los motores modernos compilan el código por partes mientras se ejecuta (compilación JIT) para que corra más rápido. Para quien programa, se trabaja igual que con un lenguaje interpretado.

Característica	C	JavaScript	TypeScript
Traducción	Compilado (gcc genera un .exe)	Interpretado por el navegador	Se traduce a JavaScript (tsc)
Tipos	Se declaran: int edad	No se declaran: let edad	Se declaran: let edad: number
Dónde corre	En el sistema operativo	En el navegador o en Node.js	Donde corra el JavaScript que genera
En el curso	Módulo anterior	UA1, UA2 y UA3	UA4, con Angular

4. Tipado dinámico y débil

En JavaScript el tipo lo tiene el **valor**, no la variable: por eso no se escribe int ni char como en C.

Dinámico: la misma variable puede guardar un número y después un texto.

Ejemplo · tipado dinámico

```
let dato = 68000;      // ahora es un número
dato = "Panadería";  // ahora es un texto: no da error
```

Débil: cuando se mezclan tipos, JavaScript convierte uno de ellos por su cuenta, sin avisar.

Ejemplo · tipado débil

```
console.log("68000" + 10);  // 6800010: el + une textos
console.log("68000" - 10);  // 67990: el - convierte a número
```

> Esta libertad es cómoda, pero causa **errores de semántica difíciles de ver**. Es una de las razones por las que existe TypeScript.

5. Un lenguaje multiparadigma

Un **paradigma** es una forma de organizar un programa. JavaScript admite cuatro y se pueden mezclar en el mismo archivo; C, en cambio, es casi solo imperativo.

Paradigma	Idea	Se ve en
Imperativo	Instrucciones paso a paso que cambian los datos, como en C.	S4 a S7
Orientado a objetos (prototipos)	Datos y acciones agrupados en objetos.	S9
Funcional	Las funciones son valores: se guardan y se pasan a otras funciones.	S7 y S8
Orientado a eventos	El código espera un clic o un envío y entonces responde.	S11 (UA2)

Imperativo: paso a paso

Ejemplo · imperativo

```
let total = 0;           // empieza en cero
total = total + 68000;   // suma el taller de cocina
total = total + 95000;   // suma el de panadería
console.log(total);      // 163000
```

Orientado a objetos basado en prototipos

Un objeto se puede escribir directamente, sin molde. Que sea «basado en prototipos» significa que un objeto hereda de **otro objeto**, no de una clase. La palabra **class** existe desde 2015 como una forma más cómoda de escribir lo mismo.

Ejemplo · objeto

```
const taller = { nombre: "Panadería", precio: 95000 };
console.log(taller.precio);  // 95000
```

Funcional: datos nuevos sin modificar los originales

Ejemplo · funcional

```
const precios = [68000, 95000, 82000];
const rebajados = precios.map((p) => p - p * 0.1);
// [61200, 85500, 73800] y precios no cambia
```

Orientado a eventos: decide la persona usuaria

Ejemplo · evento

```
formulario.addEventListener("submit", () => {
  console.log("Inscripción recibida");
});
```

6. Sintaxis, semántica y errores

La **sintaxis** son las reglas de escritura; la **semántica** es lo que el código realmente hace. Un programa puede estar bien escrito y aun así calcular otra cosa.

Tipo de error	Ejemplo	Qué pasa
Sintaxis	`console.log("Hola);`	No se ejecuta ninguna línea del archivo. La consola muestra SyntaxError.
Ejecución	`console.Log("Hola");`	Se ejecuta hasta esa línea y ahí se detiene. La consola muestra TypeError o ReferenceError.
Semántica	`68000 + 72000 / 2`	Se ejecuta sin errores, pero da 104000 en lugar del promedio, 70000. Nadie avisa.
Carga	`src="js/fichas.js"`	El archivo no existe en esa ruta. La consola muestra 404 y el script no se ejecuta.

> **Cómo leer un error**
> Un error de la consola se lee en tres partes: **tipo**, **mensaje** y **archivo:línea**. Un error de semántica solo se descubre probando con un dato cuyo resultado ya se conoce.

7. Cómo llega el código a la página

El código se escribe en un archivo .js y se enlaza con la etiqueta **script**, igual que la hoja de estilos se enlaza con **link**.

Dos formas de enlazar un archivo .js

```
<!-- Opción 1: al final del body -->
  <script src="js/app.js"></script>
</body>

<!-- Opción 2: en el head, con defer -->
  <script src="js/talleres.js" defer></script>
</head>
```

1. El navegador lee el HTML de arriba hacia abajo.
2. Al llegar a un script sin defer, lo descarga y lo ejecuta en ese momento.
3. Un script con **defer** espera a que la página termine de leerse.
4. Los resultados y los errores se ven en la **consola**: tecla F12, pestaña Console.

La consola es solo para quien programa: la persona usuaria no la ve. Allí se usan **console.log** para mensajes, **console.warn** para avisos (amarillo) y **console.error** para errores (rojo).

8. Glosario rápido

Término	Significado
ECMAScript	Estándar que define cómo funciona JavaScript.

Término	Significado
Intérprete	Programa que ejecuta el código directamente; en la web, el navegador.
Compilador	Programa que traduce todo el código antes de ejecutarlo, como gcc en C.
Motor	Parte del navegador que ejecuta JavaScript (V8, SpiderMonkey).
Tipado dinámico	El tipo lo tiene el valor y puede cambiar durante el programa.
Tipado débil	JavaScript convierte tipos por su cuenta al mezclarlos.
Paradigma	Forma de organizar un programa.
Prototipo	Objeto del que otro objeto hereda sus datos y acciones.
Evento	Algo que ocurre en la página: un clic, un envío, la carga.
Consola	Panel de F12 donde se ven mensajes y errores.
Node.js	Programa que ejecuta JavaScript fuera del navegador.